Nexa TEAM ECOBYTE

SDC HACKATHON 2026 · HOSTED BY XMUM AI CLUB

RunPod
Proof of Use

How Nexa uses a RunPod Serverless GPU endpoint: the workload, the API, and the worker code. Participant Handbook §10.7.

TEAM
EcoByte

PROJECT
Nexa

DOMAIN
Domain 1 · Aiversecity Future-Ready Workforce

SDG TRACK
SDG 4 · Target 4.4 (+5, 8, 9, 10)

LIVE PROTOTYPE
https://nexa-xxx.vercel.app/

FILE
SDC2026_EcoByte_Runpod_POU.pdf

What runs on RunPod

Workload: semantic skill closeness. Nexa's role score is rule-based so every point can be checked by hand. RunPod adds one GPU signal beside it: how close, in meaning, a person's skills are to each target role's skills. The open-source embedding model **BAAI/bge-small-en-v1.5** (MIT) turns every skill into a vector on the GPU; for each skill a role needs, the endpoint finds the person's nearest skill and averages those similarities.

Why a GPU: one request embeds every profile skill and every skill of all 21 roles at once; batching this on CUDA keeps it fast, and serverless scale-to-zero means we only pay while it runs.

Safety: only skill names are sent (no names, contact details or CV text). The result is shown as context and never changes the explainable score. Without RunPod, Nexa hides this one indicator and everything else works.

Endpoint type	RunPod Serverless (queue-based worker)
Endpoint ID / URL	Kept private in Vercel environment variables (RUNPOD_ENDPOINT_ID) `https://api.runpod.ai/v2/<ENDPOINT_ID>/runsync`
GPU type	Serverless CUDA GPU worker (16 GB class is enough for this model)
Scaling	Scale-to-zero: billed only while a request runs
Container image	Built from `runpod/Dockerfile` (runpod/pytorch 2.4.0, CUDA 12.4.1, Python 3.11); model baked in
Called from	`app/api/semantic/route.ts` on Vercel; key and endpoint ID in environment variables

Flow: browser → Nexa engine on Vercel → `/api/semantic` (server-side key) → RunPod Serverless GPU → scores back to the role cards.

Request and response

```
curl -X POST "https://api.runpod.ai/v2/$RUNPOD_ENDPOINT_ID/runsync" \
  -H "Authorization: Bearer $RUNPOD_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"input": {
        "profile": ["Customer care", "Empathy", "Communication"],
        "roles": {
          "training_specialist": ["Instructional design", "Communication"],
          "data_analyst": ["Data analysis", "SQL"]
        }}}'
```

```
{ "status": "COMPLETED",
  "output": { "model": "BAAI/bge-small-en-v1.5", "device": "cuda",
              "scores": { "training_specialist": 0.71, "data_analyst": 0.48 } } }
```

Example values for illustration; real scores depend on the profile.

Worker code (runpod/handler.py, excerpt)

```
MODEL_ID = "BAAI/bge-small-en-v1.5"
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
model = SentenceTransformer(MODEL_ID, device=DEVICE)

def handler(job):
    profile = job["input"]["profile"]; roles = job["input"]["roles"]
    mine = model.encode(profile, convert_to_tensor=True, normalize_embeddings=True)
    scores = {}
    for role_id, skills in roles.items():
        theirs = model.encode(skills, convert_to_tensor=True, normalize_embeddings=True)
        best = (theirs @ mine.T).max(dim=1).values   # nearest skill per need
        scores[role_id] = round(float(best.mean()), 4)
    return {"model": MODEL_ID, "device": DEVICE, "scores": scores}

runpod.serverless.start({"handler": handler})
```